

임베디드 소프트웨어

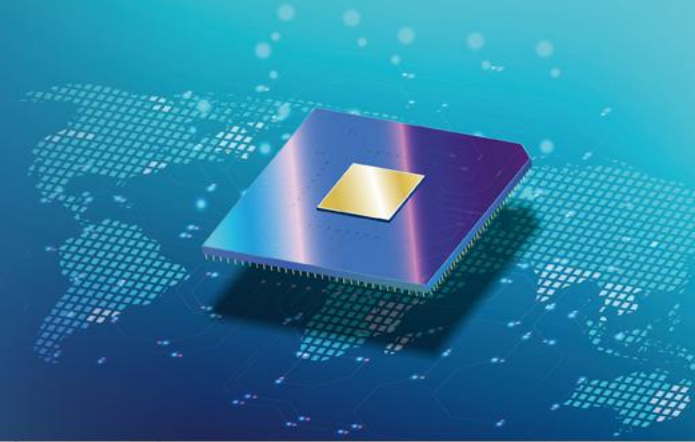
서울대학교 차세대반도체 혁신융합대학





CHAPTER 4

Input & Output (I/O) Ports



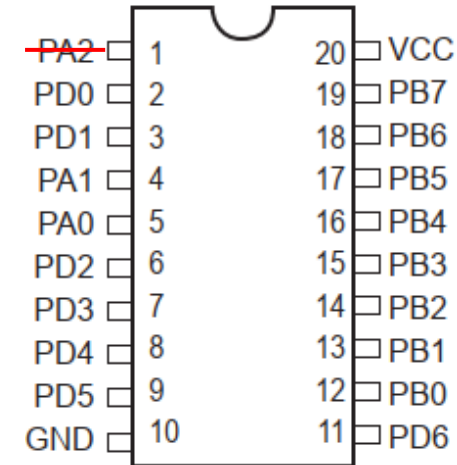
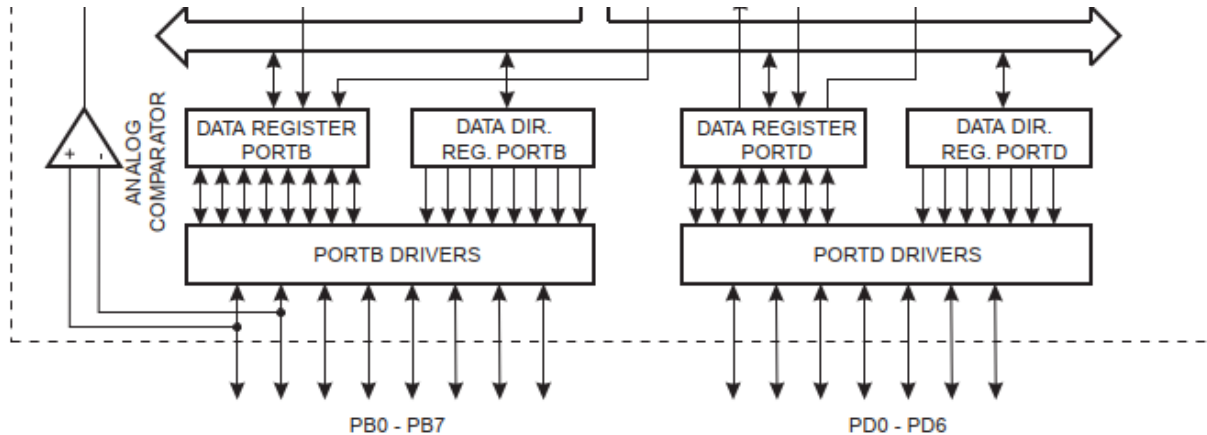
ATtiny4313 GPIO Ports

GPIO: General Purpose Input Output

ATtiny4313 에는 Port A, Port B, Port D 가 있음.

- Port A 핀: PA0 ~ PA2
 - Port B 핀: PB0 ~ PB7
 - Port D 핀: PD0 ~ PD6
- 총 18개 Pin

PA2: 사용 불가 (RESET 용도)



각 Port는 bit 단위로 Input 또는 Output Pin으로 동작

- DDRx register에서 Input / Output Mode를 설정.
- PORTx register를 통해 데이터를 출력함 (write).
- PINx register를 통해 데이터를 입력 받음 (read).

x = A, B 또는 D

- 출력 register (PORTx)에 값을 쓰면 핀의 level (high 또는 low)는 계속 유지됨.
- 입력 register (PINx)는 매 순간의 핀 level을 나타냄.

ATtiny4313 GPIO Registers (Port B)

Datasheet Page 70

DDRB (Port B Data Direction Register)

Bit	7	6	5	4	3	2	1	0
0x17 (0x37)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

Bit를 0으로 설정하면 그 핀은 Input mode로 동작하고,
Bit를 1로 설정하면 그 핀은 Output mode로 동작.

PORTB (Port B Data Register)

Bit	7	6	5	4	3	2	1	0
0x18 (0x38)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

Output Mode 인 경우:

Bit를 0으로 설정하면 그 핀에 0 (0V)가 출력되고,
Bit를 1로 설정하면 그 핀에 1 (5V)가 출력됨.

[특수 용도] Input Mode 인 경우

Bit를 1로 설정하면 그 핀에 내부 Pull-Up* 저항이 활성화 됨.

PINB (Port B Input Register)

Bit	7	6	5	4	3	2	1	0
0x16 (0x36)	PINB7	PINB6	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

Input Mode 인 경우:

Bit 번호에 해당하는 핀의 논리값 (0 또는 1) 이 Bit에 나타남.

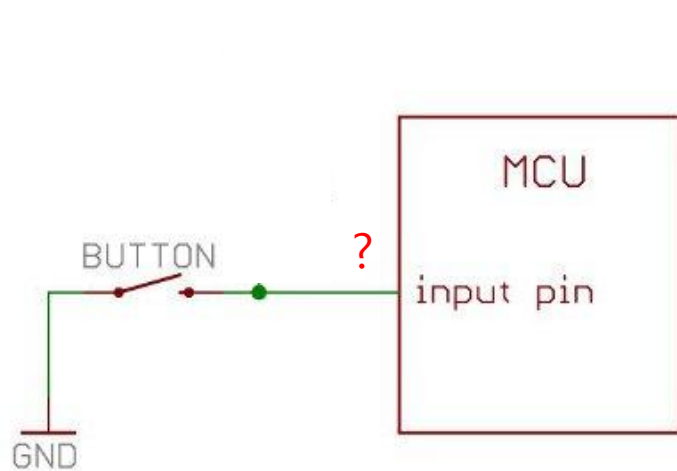
[특수 용도] Output Mode 인 경우

Bit를 1로 설정하면 그 핀의 출력 값이 반전 됨 (0→1, 1→0).

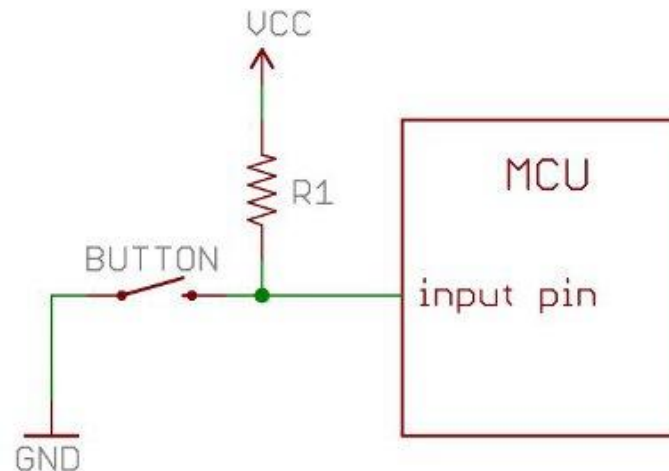
Register 이름과 Bit 이름을 C code 에서 사용할 수 있음.

* Pull-Up: 입력 핀을 저항을 통해 VCC에 연결시켜, 입력 신호가 없는 경우 1로 인식되도록 하는 회로.

Pull-Up Resistor



I/O Pin이 Input으로 설정된 경우, pull-up 저항이 없으면 스위치가 눌리지 않으면 핀의 값이 High 인지 Low 인지 알 수 없음.



Input 핀에 pull-up 저항을 연결하면, 스위치를 누르지 않는 상태에는 High 상태가 유지됨.



스위치를 누르면 Low.

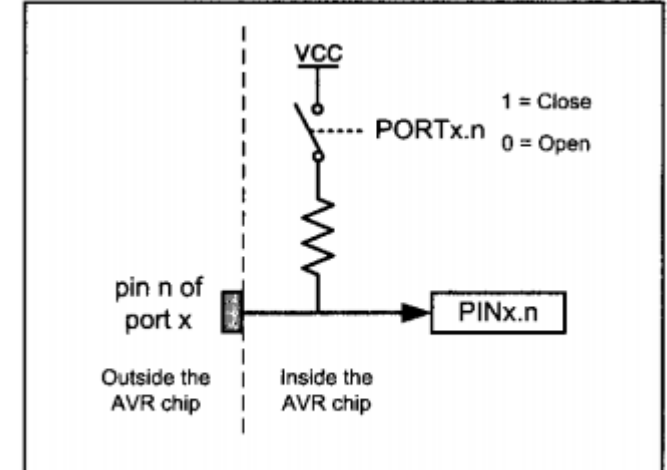


Figure 4-4. The Pull-up Resistor

AVR MCU는 MCU 내부에 pull-up 저항 회로가 있어서, 활성화 시키면 외부에 저항을 연결하지 않아도 됨.

ATtiny4313 GPIO Registers (Port D, A)

DDRD (Port D Data Direction Register)

Bit	7	6	5	4	3	2	1	0
0x11 (0x31)	–	DDD6	DDD5	DDD4	DDD3	DDD2	DDD1	DDD0
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

PORTD (Port D Data Register)

Bit	7	6	5	4	3	2	1	0
0x12 (0x32)	–	PORTD6	PORTD5	PORTD4	PORTD3	PORTD2	PORTD1	PORTD0
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

PIND (Port D Input Register)

Bit	7	6	5	4	3	2	1	0
0x10 (0x30)	–	PIND6	PIND5	PIND4	PIND3	PIND2	PIND1	PIND0
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

Port D Register 와 동일하지만, Bit 0~6 만 있음.

DDRA (Port A Data Direction Register)

Bit	7	6	5	4	3	2	1	0
0x1A (0x3A)	–	–	–	–	–	DDA2	DDA1	DDA0
Read/Write	R	R	R	R	R	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

PORTA (Port A Data Register)

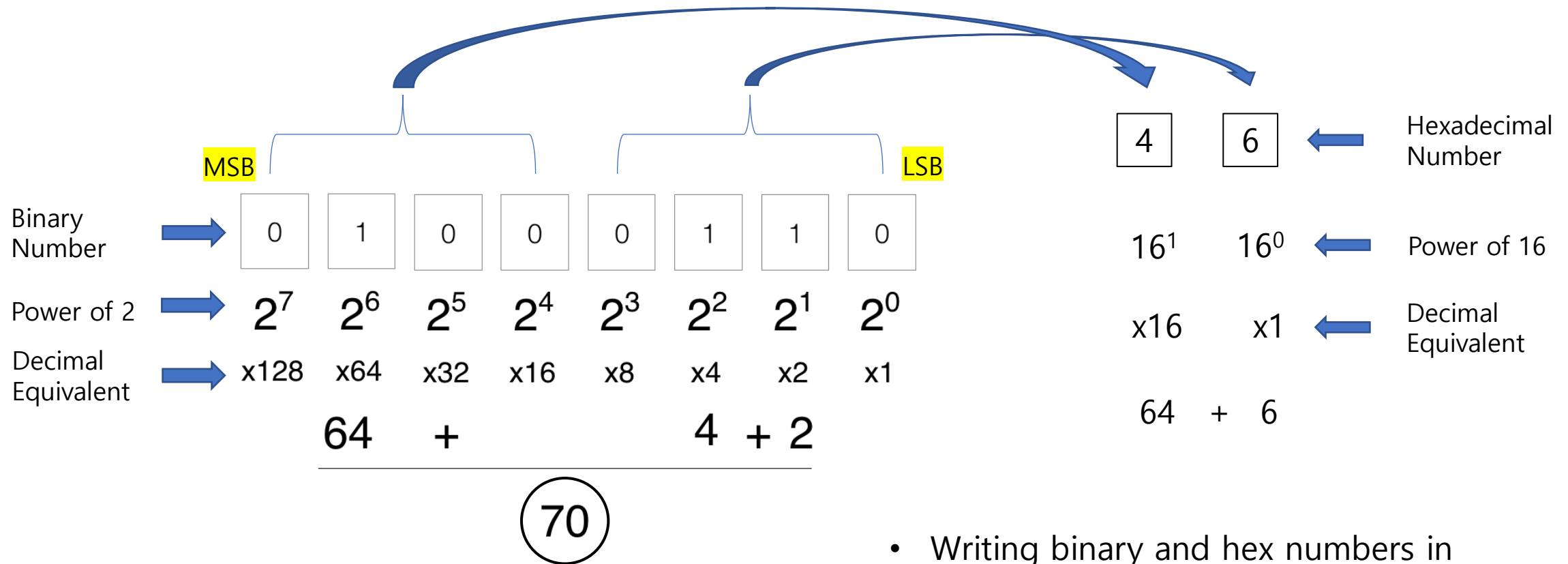
Bit	7	6	5	4	3	2	1	0
0x1B (0x3B)	–	–	–	–	–	PORTA2	PORTA1	PORTA0
Read/Write	R	R	R	R	R	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

PINA (Port A Input Register)

Bit	7	6	5	4	3	2	1	0
0x19 (0x39)	–	–	–	–	–	PINA2	PINA1	PINA0
Read/Write	R	R	R	R	R	R/W	R/W	R/W
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

Port D Register 와 동일하지만, Bit 0~2 만 있음.

2진수와 16진수 체계



- MSB = Most Significant Bit
- LSB = Least Significant Bit

- Writing binary and hex numbers in C and Python:
 - 0b01000110 (binary)
 - 0x46 (hexadecimal)

음수 (Negative Number)의 표현

Sign-Magnitude

Sign Magnitude

↓

$$\begin{aligned} +34 &= 0\ 0\ 1\ 0\ 0\ 0\ 1\ 0 \\ -34 &= 1\ 0\ 1\ 0\ 0\ 0\ 1\ 0 \end{aligned}$$

1's Complement

$$\begin{aligned} +34 &= 0\ 0\ 1\ 0\ 0\ 0\ 1\ 0 \\ &\quad \downarrow \downarrow \downarrow \downarrow \downarrow \downarrow \downarrow \downarrow \\ -34 &= 1\ 1\ 0\ 1\ 1\ 1\ 0\ 1 \end{aligned}$$

2's Complement

$$\begin{aligned} +34 &= 0\ 0\ 1\ 0\ 0\ 0\ 1\ 0 \\ &\quad \downarrow \downarrow \downarrow \downarrow \downarrow \downarrow \downarrow \downarrow \\ &\quad 1\ 1\ 0\ 1\ 1\ 1\ 0\ 1 \\ &\quad + \qquad \qquad \qquad 1 \\ &\quad \hline -34 &= 1\ 1\ 0\ 1\ 1\ 1\ 1\ 0 \end{aligned}$$

컴퓨터 시스템에서는 2's Complement를 사용함.

Data Types and Sizes

- C 언어에서는 data type의 size가 정해져 있지 않음.
 - 유일한 정의는 $\text{char} \leq \text{short} \leq \text{int} \leq \text{long}$
- C compiler, Processor Architecture마다 data type의 size를 정함.

gcc: GNU Project가 개발한 open-source C compiler.

avr-gcc: AVR architecture를 위한 gcc.

avr-gcc의 data size

Data Type	Size	Data Range
unsigned char	8-bit	0 to 255
char	8-bit	-128 to +127
unsigned int	16-bit	0 to 65,535
int	16-bit	-32,768 to +32,767
unsigned long	32-bit	0 to 4,294,967,295
long	32-bit	-2,147,483,648 to +2,147,483,648

gcc는 data width를 명시적으로 지정하는 data type도 정의하고 있음.

`uint8_t`

`int8_t`

`uint16_t`

`int16_t`

`uint32_t`

`int32_t`

같은 size의 변수라도 signed 또는 unsigned로 선언함에 따라 값의 해석이 달라짐.

`uint8_t`

1	1	0	1	1	0	0	1
---	---	---	---	---	---	---	---

217 (0xD9)

`int8_t`

1	1	0	1	1	0	0	1
---	---	---	---	---	---	---	---

-39 (0xD9)

Bitwise Operators

Truth Table

		OR	AND	XOR	NOT
A	B	A B	A & B	A ^ B	~A
0	0	0	0	0	1
0	1	1	0	1	1
1	0	1	0	1	0
1	1	1	1	0	0

Examples

Operators	Description	Example	Result
	Bitwise OR	4 2	6
&	Bitwise AND	1 & 3	1
^	Bitwise exclusive OR	4 ^ 3	7
<<	Bitwise Left Shift	2 << 1	4
>>	Bitwise Right Shift	4 >> 1	2
~	Bitwise Not	~6	-7

Bitwise Operation Examples

Operators	Description	Example	Result
	Bitwise OR	4 2	6
&	Bitwise AND	1&3	1
^	Bitwise exclusive OR	4^3	7
<<	Bitwise Left Shift	2<<1	4
>>	Bitwise Right Shift	4>>1	2
~	Bitwise Not	~6	-7

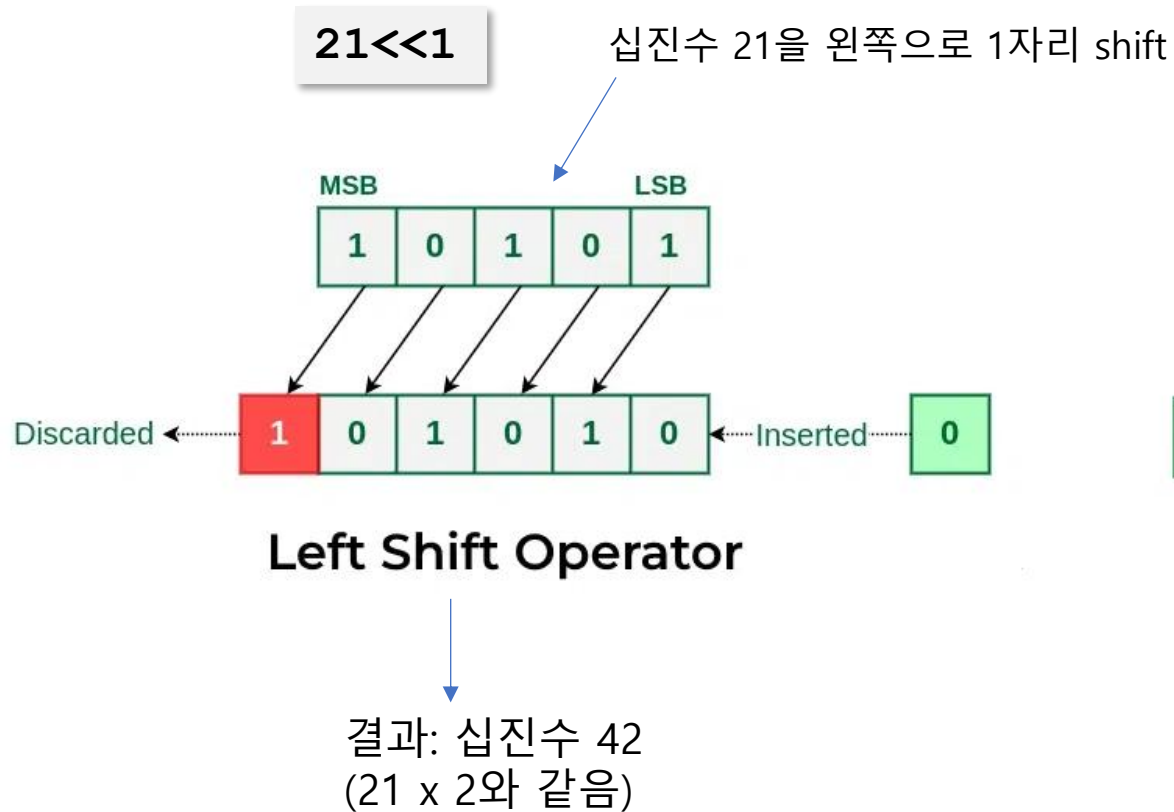
십진수 4 → 0 1 0 0
 십진수 2 → 0 0 1 0
 OR
 0 1 1 0 → 십진수 6

십진수 1 → 0 0 0 1
 십진수 3 → 0 0 1 1
 AND
 0 0 0 1 → 십진수 1

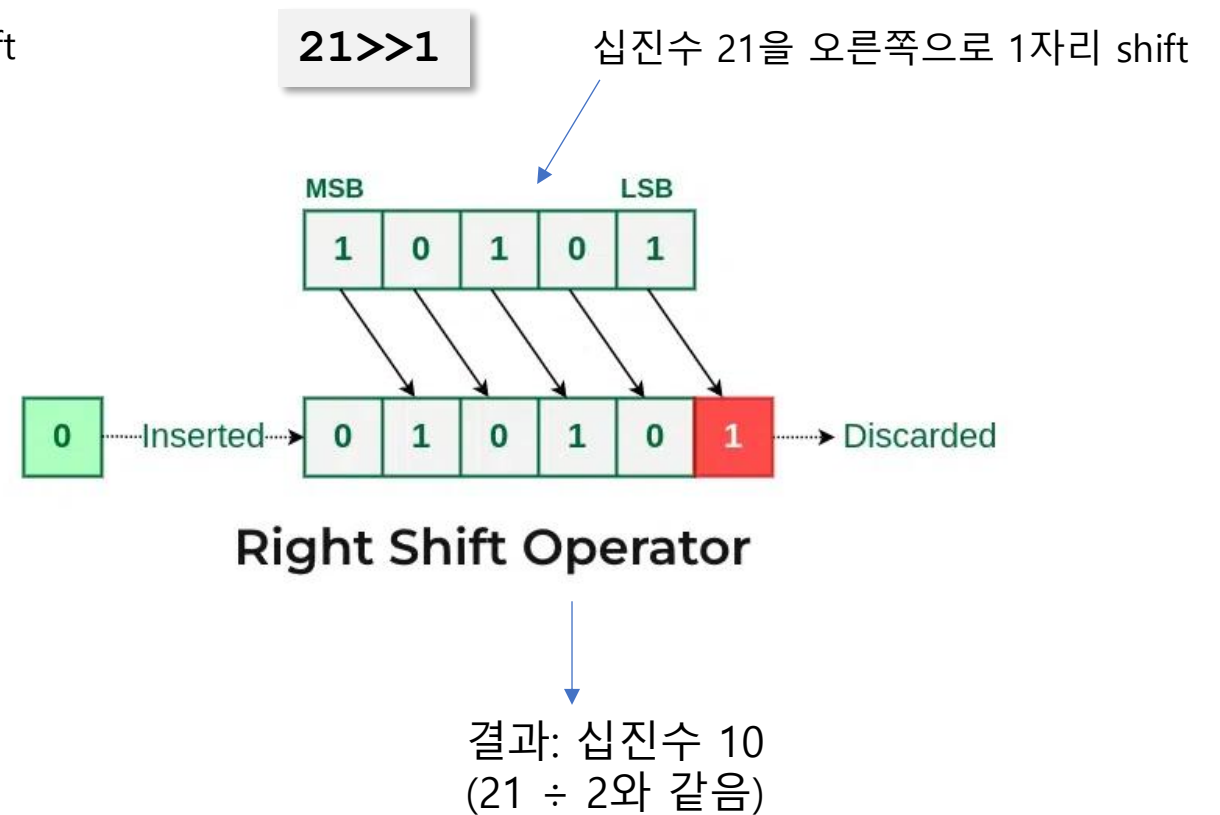
십진수 4 → 0 1 0 0
 십진수 3 → 0 0 1 1
 XOR
 0 1 1 1 → 십진수 7

십진수 6 → 0 1 1 0
 NOT
 1 0 0 1 → 십진수 -7

Logical (Unsigned) Shift Operations



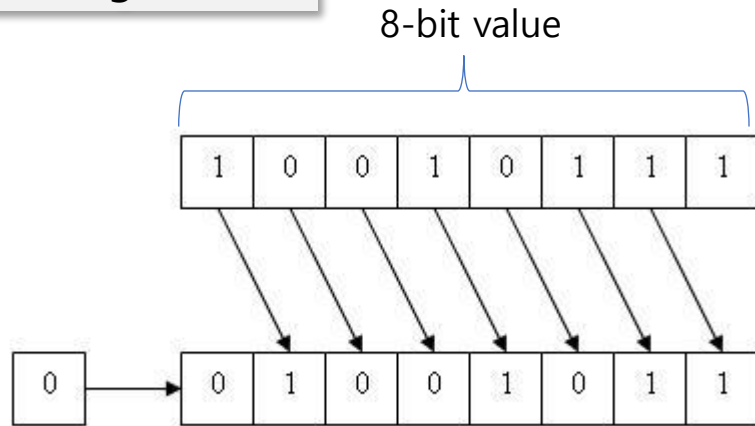
n자리 left shift는 2^n 을 곱하는 것과 같음.



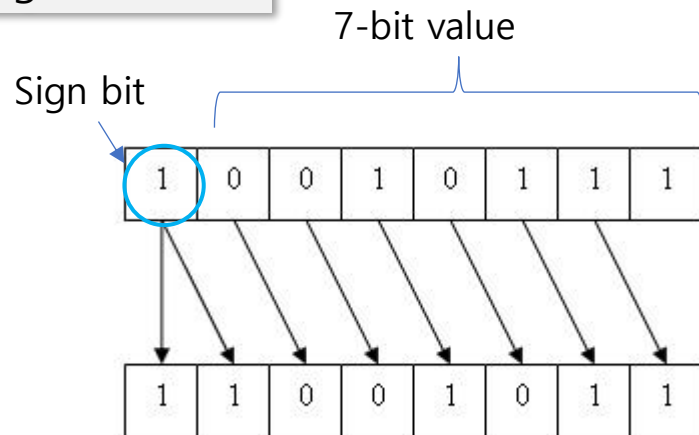
n자리 right shift는 2^n 으로 나누는 것과 같음.

Arithmetic (Signed) Shift Operations

Unsigned Right Shift



Signed Right Shift



unsigned int

$$\begin{array}{|c|c|c|c|c|c|c|c|} \hline 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 \\ \hline \end{array} \gg 2 = \begin{array}{|c|c|c|c|c|c|c|c|} \hline 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \\ \hline \end{array}$$

217 (0xD9) 54 (0x36)

signed int

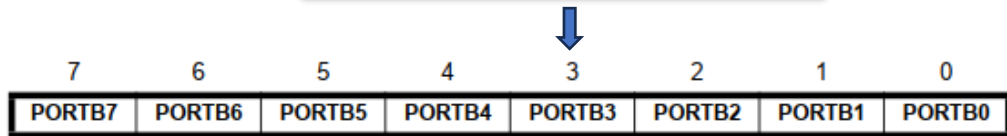
$$\begin{array}{|c|c|c|c|c|c|c|c|} \hline 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 \\ \hline \end{array} \gg 2 = \begin{array}{|c|c|c|c|c|c|c|c|} \hline 1 & 1 & 1 & 1 & 0 & 1 & 1 & 0 \\ \hline \end{array}$$

-39 (0xD9) -10 (0xF6)

★ 변수를 signed 또는 unsigned type으로 선언함에 따라 signed 또는 unsigned 연산이 적용됨.

상수 1 Bit를 출력하는 C 구문

Bit 3을 1로 설정하는 경우



다른 Bit의 값은
변경하면 안 됨.

`PORTB = PORTB | 0b00001000`

`PORTB |= 0b00001000`

동일한 구문.

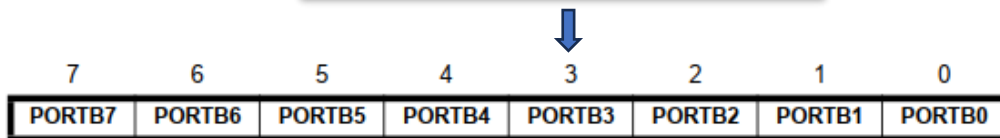
`PORTB |= (1<<3)`

동일한 구문.

`PORTB |= (1<<PORTB3)`

동일한 구문
(io.h header 파일에서 정의
되어 있음).

Bit 3을 0으로 설정하는 경우



다른 Bit의 값은
변경하면 안 됨.

`PORTB = PORTB & 0b11110111`

`PORTB &= 0b11110111`

동일한 구문.

`PORTB &= ~0b00001000`

동일한 구문.

`PORTB &= ~(1<<3)`

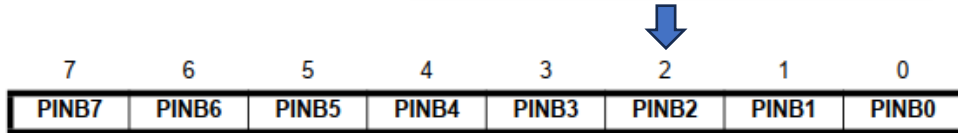
동일한 구문.

`PORTB &= ~(1<<PORTB3)`

동일한 구문.

1 Bit를 입력 받는 C 구문

Bit 2의 값을 읽으려는 경우



방법 ①

`Result = PINB & 0b00000100`

PINB2 값만 유지되고, 나머지 bits 는 모두 0 이 됨.

- PB2 핀이 0 이면 Result=0
- PB2 핀이 1 이면 Result=4.

`Result = PINB & (1<<2)`

동일한 구문.

`Result = PINB & (1<<PINB2)`

동일한 구문.

방법 ②

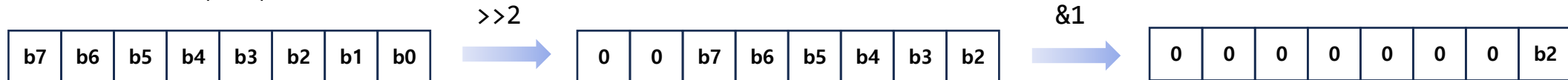
`Result = (PINB>>2) & 1`

PINB2 값을 Bit 0 자리로 이동시키고, 1과 AND 하여 PINB2 값만 추출함.

- PB2 핀이 0 이면 Result=0
- PB2 핀이 1 이면 Result=1.

`Result = (PINB>>PINB2) & 1`

동일한 구문.



변수를 출력하는 C 구문

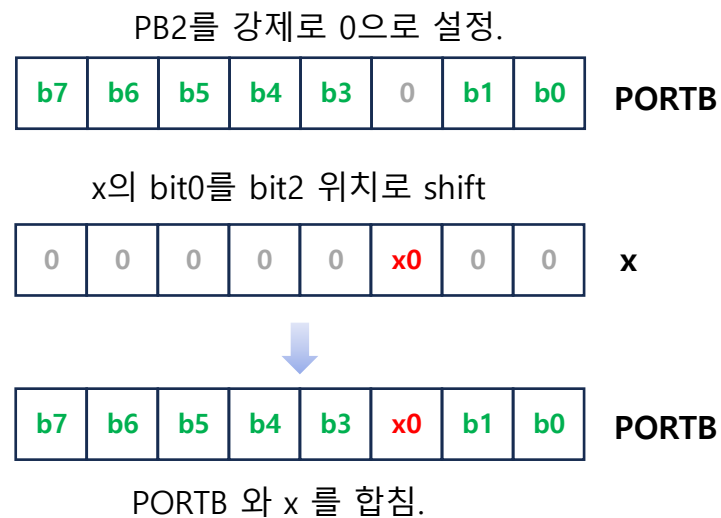
(예) 변수 x의 bit 0을, PORTB의 bit 2로 출력할 때

방법 1:

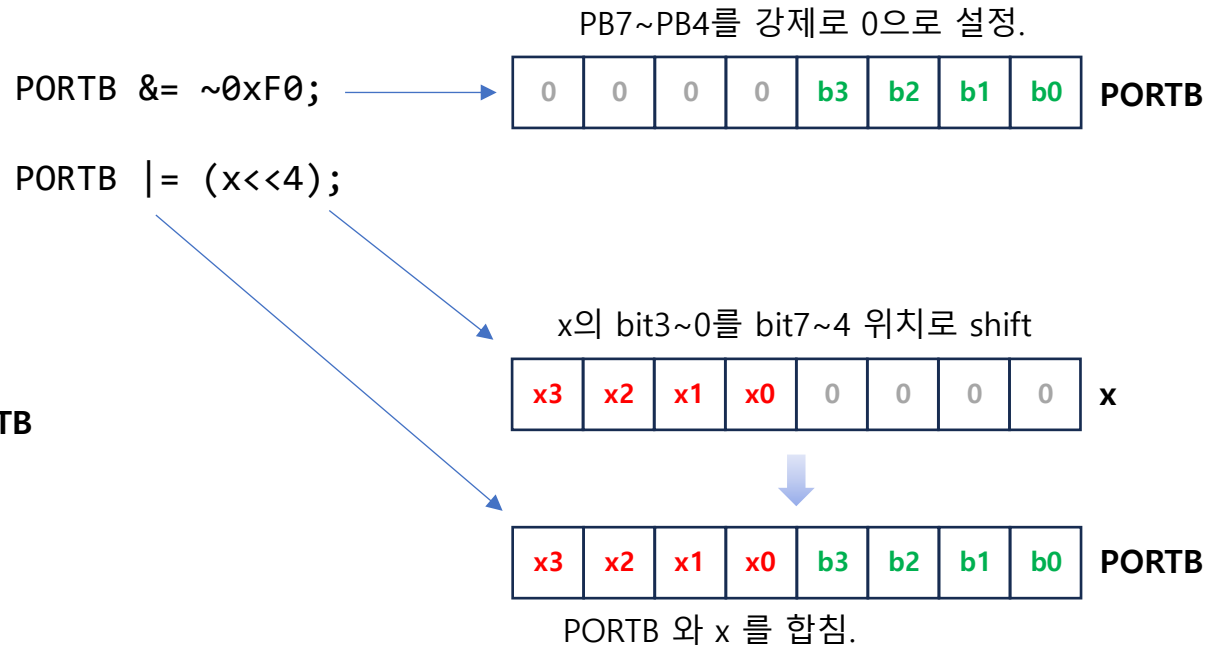
```
if ((x&1)==0)
    PORTB &= ~(1<<2);
else
    PORTB |= (1<<2);
```

방법 2:

```
PORTB &= ~(1<<2);
PORTB |= ((x&1)<<2);
```



(예) 변수 x의 bit 0~3을, PORTB의 bit 4~7로 출력할 때



Keywords: static, inline, const

static 함수

C 함수는 기본적으로 전역 (global) 함수라서 모든 파일에서 호출할 수 있으나, static 함수는 정의된 파일에서만 호출할 수 있음.

static 전역 변수

일반 전역 변수는 모든 파일에서 사용할 수 있음.

- ① 정의된 파일에서만 사용할 수 있음.
- ② Data Segment에 할당되어 프로그램 종료까지 존재하면서 그 값을 유지함.

static 지역 변수

- ① 정의된 함수 안에서만 사용할 수 있음.
- ② Data Segment에 할당되어 프로그램 종료까지 존재하면서 그 값을 유지함.

일반 지역 변수는 stack에 할당되며, 함수가 시작할 때 생성되었다가 함수가 종료하면 소멸함.

inline 함수

별도의 함수 코드를 생성하여 그 코드를 호출하는 대신, 호출하는 코드 내부에 함수 코드가 섞여서 하나의 코드로 컴파일 됨.

C Macro 와 유사.

const 키워드

변수에 const 키워드를 붙이면, 컴파일러는 이 변수가 항상 같은 값을 유지하는 상수로 인식함 (즉 read only).

volatile 키워드

변수에 volatile 키워드를 붙이면, 그 변수는 코드의 흐름과 상관 없이 변할 수 있음을 알려줌. 컴파일러는 volatile 변수의 최적화를 하지 않고, 항상 메모리에 있는 값을 읽음.

LED Blink Code

```
#define F_CPU 8000000 ← Clock=8MHz를 알려줌.

#include <avr/io.h> ← I/O Port 이름을 사용하기 위해 include.
#include <util/delay.h> ← _delay_ms() 함수를 사용하기 위해 include.

int main(void) {
    DDRB |= 0b00000010; // DDRB Bit 1 ← 1

    while (1) {
        PORTB |= 0b00000010; // PORTB Bit 1 ← 1
        _delay_ms(500); // Wait 500ms

        PORTB &= 0b11111101; // PORTB Bit 1 ← 0
        _delay_ms(500); // Wait 500ms
    }
}
```

DDRB	Bit	7	6	5	4	3	2	1	0
	0x17 (0x37)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0
	Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
	Initial Value	0	0	0	0	0	0	0	0
PORTB	Bit	7	6	5	4	3	2	1	0
	0x18 (0x38)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0
	Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
	Initial Value	0	0	0	0	0	0	0	0

LED Blink Code 단순화 (PINB 반전 기능 사용)

```
#define F_CPU 8000000

#include <avr/io.h>
#include <util/delay.h>

int main(void) {
    DDRB |= 0b00000010;    // DDRB Bit 1 ← 1

    while (1) {
        PINB = 0b00000010; // PINB Bit 1 ← 1
        _delay_ms(500);     // Wait 500ms
    }
}
```

★ PINB에 Write하는 경우에는 OR (|) 연산을 하지 않음.

PINB (Port B Input)

Bit	7	6	5	4	3	2	1	0
0x16 (0x36)	PINB7	PINB6	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

[특수 용도] Output Mode 인 경우
Bit를 1로 설정하면 그 핀의 출력 값이 반전 됨 (0→1, 1→0).

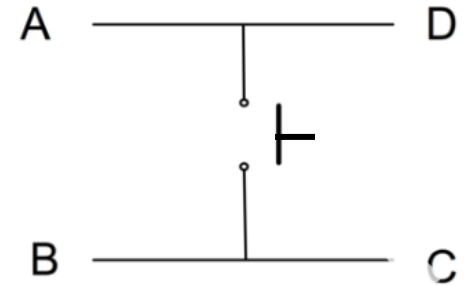
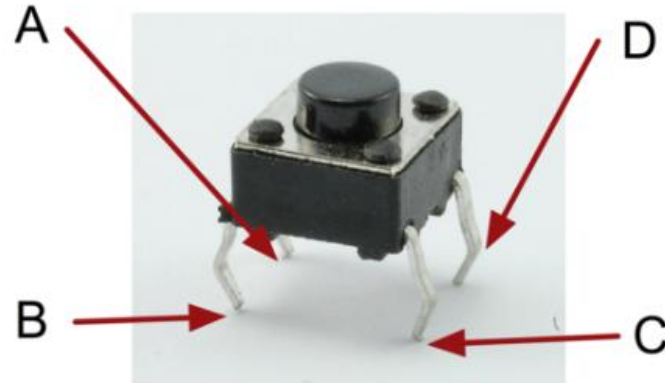
Input & Output 실습

택트 스위치를 눌렀을 때 LED가 점등

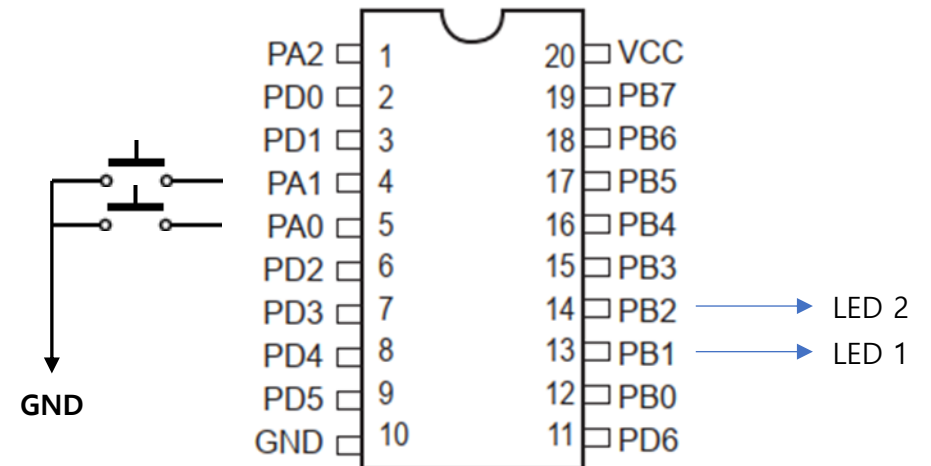
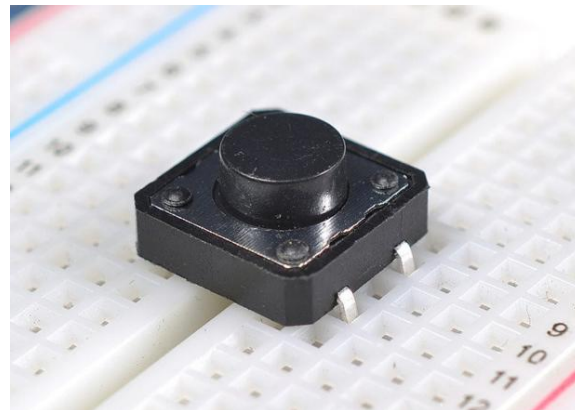
코드 예제

PA0을 Input으로 설정하고 Pull-up 저항을 활성화.

```
while (1) {  
    PA0 이 0 이면 LED 1을 ON,  
    아니면 OFF.  
}
```



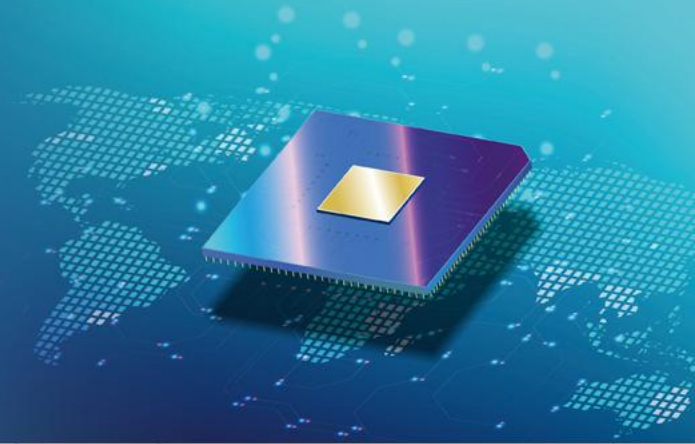
A와D, B와C 는 내부에서 연결되어 있음.





CHAPTER 5

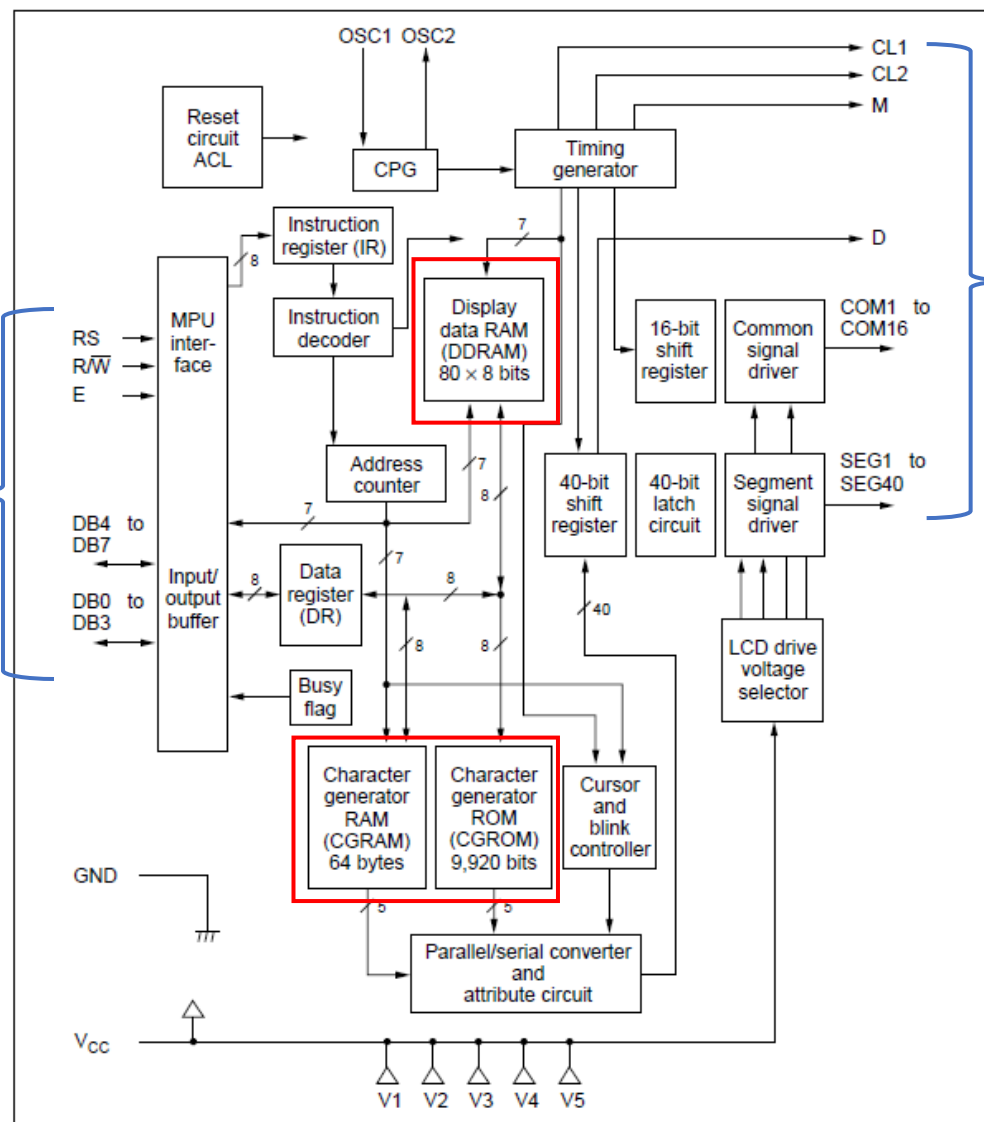
Character LCD



HD44780 LCD Controller

MCU에 연결
하는 핀

내부 메모리



LCD 화면에
연결하는 핀

HD44780 IC가 내장된 LCD 모듈

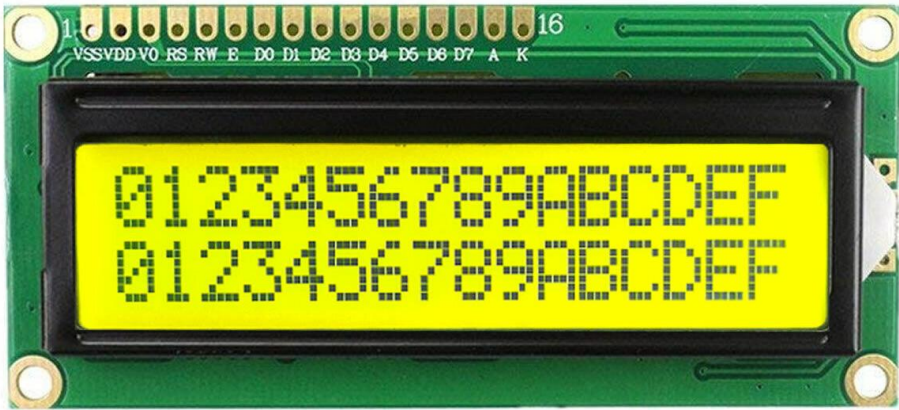


Datasheet
3쪽

DDRAM과 화면의 관계

1602 LCD 모듈

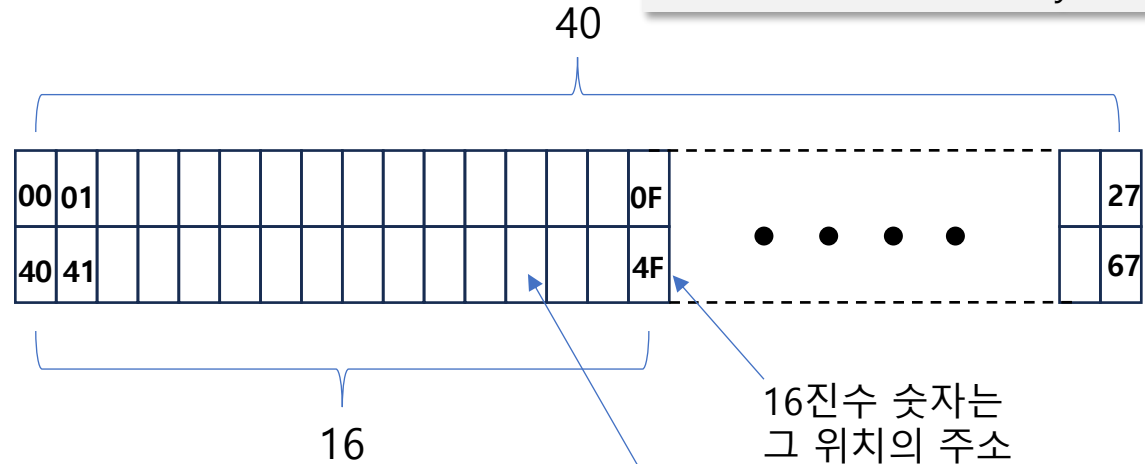
(가로 16, 세로 2)



16

Display Data RAM (DDRAM)

HD44780 IC 내부의 전체
DDRAM은 $40 \times 2 = 80$ byte



각 위치에 ASCII 코드를 Write하면 그 코드의 Font가 표시됨.

DDRAM 중 16x2 bytes 만
LCD에 표시

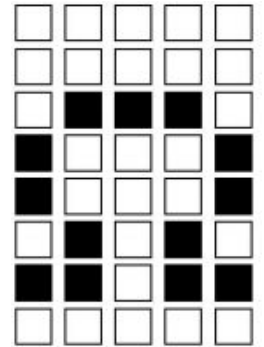
ASCII 코드와 문자 Font

ASCII 코드 (10진수로 표시하는 경우)

32	48 0	64 @	80 P	96 `	112 p
33 !	49 1	65 A	81 Q	97 a	113 q
34 "	50 2	66 B	82 R	98 b	114 r
35 #	51 3	67 C	83 S	99 c	115 s
36 \$	52 4	68 D	84 T	100 d	116 t
37 %	53 5	69 E	85 U	101 e	117 u
38 &	54 6	70 F	86 V	102 f	118 v
39 '	55 7	71 G	87 W	103 g	119 w
40 (	56 8	72 H	88 X	104 h	120 x
41)	57 9	73 I	89 Y	105 i	121 y
42 *	58 :	74 J	90 Z	106 j	122 z
43 +	59 ;	75 K	91 [	107 k	123 {
44 ,	60 <	76 L	92 \	108 l	124
45 -	61 =	77 M	93]	109 m	125 }
46 .	62 >	78 N	94 ^	110 n	126 ~
47 /	63 ?	79 O	95 _	111 o	127 DEL

원하는 위치 주소에 ASCII 코드를 write하면, 그 위치에 ASCII 코드에 해당하는 문자 font가 표시됨.

ABCDEFGHIJ
KLMNOPQRST
UVWXYZabcd



208개 ASCII 코드의 문자 font는 5x8의 점 (dot) 으로 구성되어, HD44780의 CG ROM 에 저장되어 있음.

사용자가 정의할 수 있는 8개 문자의 font를 저장하는 CG RAM 도 있음.

* ASCII = American Standard Code for Information Interchange

HD44780의 MCU 연결 핀

Data Bus

DB0~DB7: MCU가 데이터를 HD44780에 Write 하거나, HD44780로부터 Read 하는 통로.

- 양방향 (Bi-directional) 이지만 수업에서는 MCU→HD44780 방향으로만 사용.
- 8-bit mode 또는 4-bit mode 로 동작 가능.

Enable

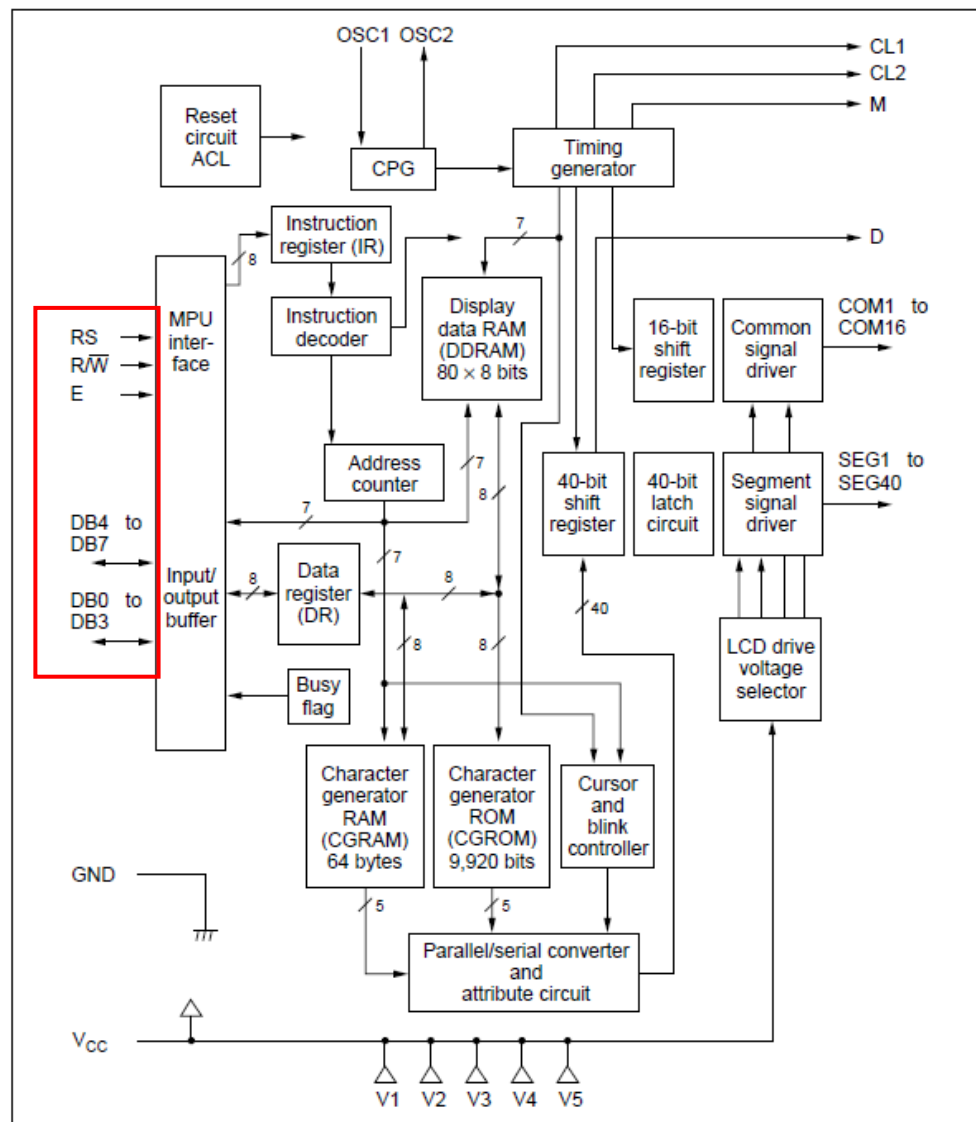
E: 0=HD44780이 반응하지 않음
1=HD44780이 DB0~7를 통해 데이터를 읽거나 내줌

Read/Write

R/W: 0=Write (MCU→HD44780) 할 것을 알려줌
1=Read (HD44780→MCU) 할 것을 알려줌

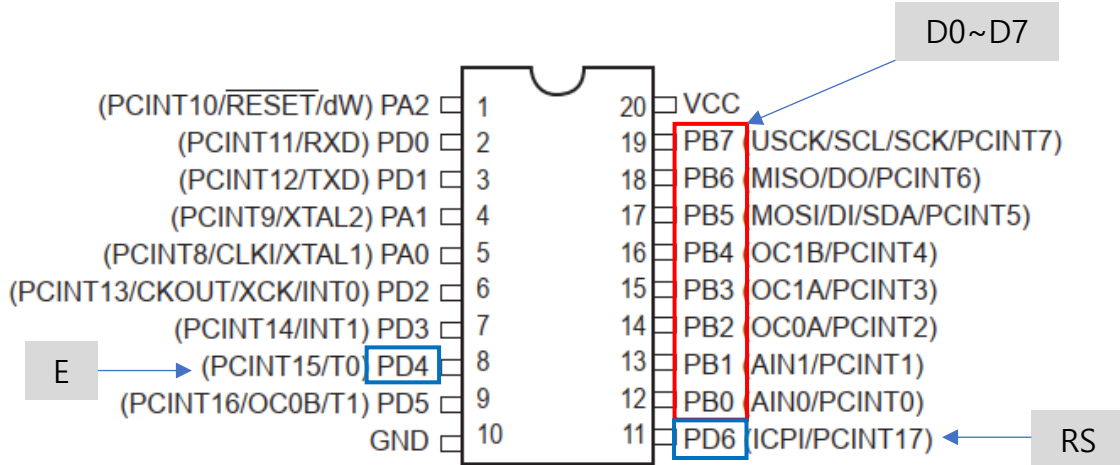
Register Select

RS: 0=Instruction Register를 선택
1=Data Register를 선택

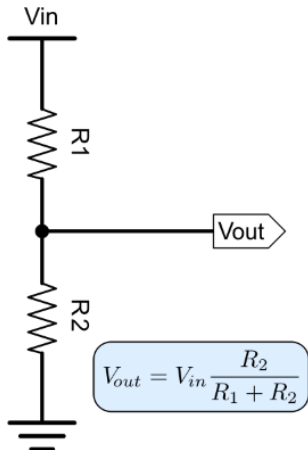


Datasheet
3쪽

HD44780와 ATtiny의 연결 (8-bit mode)

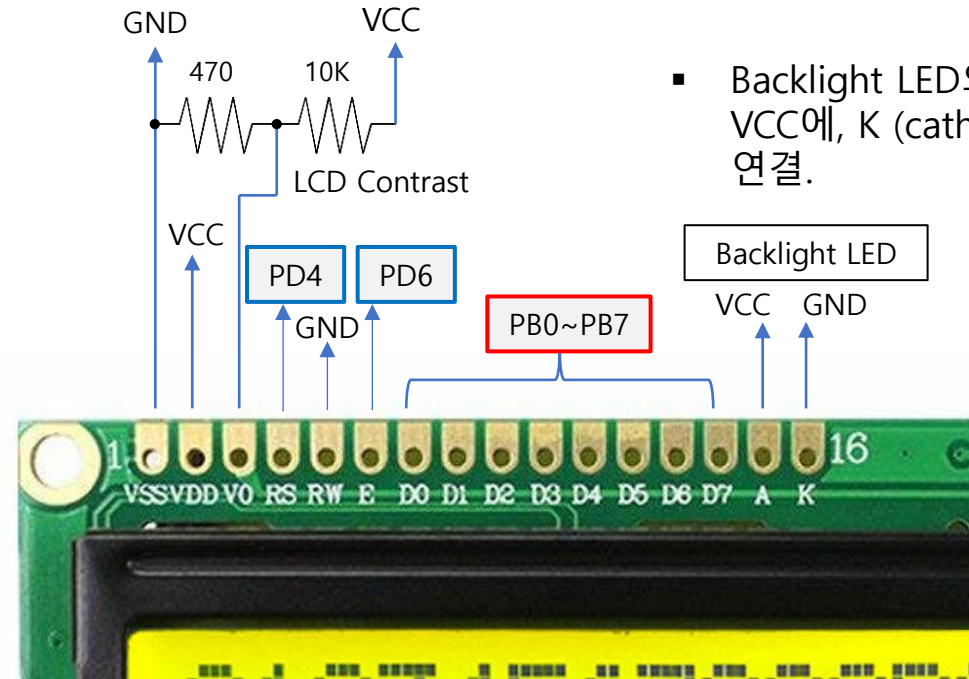


- ATtiny가 항상 데이터를 write 할 것이므로 RW 핀은 0으로 고정 (GND에 연결) → ATtiny GPIO 핀을 절약.
- VO는 LCD의 명암을 조절하는 전압을 입력하는 핀으로 2개의 저항으로 voltage divider를 만들어야 하나, 간단히 GND에 연결해도 무방.



예시

- D0~D7 핀은 PB0~PB7에 연결
- E 핀은 PD4에 연결
- RS 핀은 PD6에 연결



- Backlight LED의 A (anode)는 VCC에, K (cathode)는 GND에 연결.

HD44780 Write Sequence

DB0~DB7 전달 방향: ATtiny → HD44780

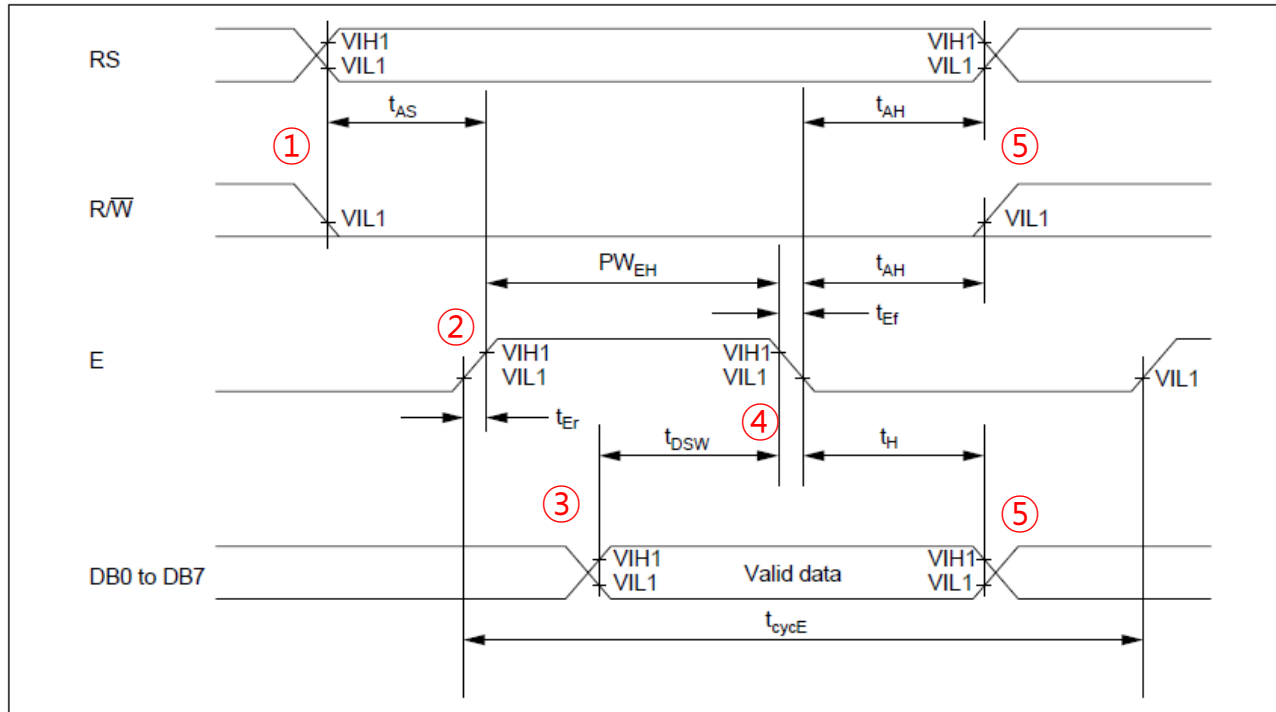


Figure 25 Write Operation

Datasheet
58쪽

Write Operation

Datasheet
49쪽

Item	Symbol	Min	Typ	Max	Unit
Enable cycle time	t_{cycE}	500	—	—	ns
Enable pulse width (high level)	PW_{EH}	230	—	—	
Enable rise/fall time	t_{Er}, t_{Ef}	—	—	20	
Address set-up time (RS, R/W to E)	t_{AS}	40	—	—	
Address hold time	t_{AH}	10	—	—	
Data set-up time	t_{DSW}	80	—	—	
Data hold time	t_H	10	—	—	

- ① RW 핀에 0을, RS 핀에 0 또는 1을 출력.
- ② t_{AS} 후 E 핀에 1을 출력.
- ③ DB0~DB7 핀에 Write할 데이터를 출력.
- ④ t_{DSW} 후 E 핀에 0을 출력 (이 때 HD44780은 DB 값을 읽음).
- ⑤ t_H 후 DB0~DB7을 꺼도 됨. t_{AH} 후 RS, R/W를 꺼도 됨.

- ATtiny가 8MHz로 동작 시 1명령 사이클은 125ns 이므로, t_{AS} , t_{DSW} , t_H , t_{AH} 보다 큼 (모두 만족함).
- 따라서 신호의 순서만 지키면 됨.

HD44780 Instructions (명령어)

Datasheet
24~25쪽

Instructions	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	실행시간
Clear Display	0	0	0	0	0	0	0	0	0	1	1.52ms
Return Home	0	0	0	0	0	0	0	0	1	-	1.52ms
Entry Mode Set	0	0	0	0	0	0	0	1	I/D	S	37us
Display On/Off	0	0	0	0	0	0	1	D	C	B	37us
Cursor/Display Shift	0	0	0	0	0	1	S/C	R/L	-	-	37us
Function Set	0	0	0	0	1	DL	N	F	-	-	37us
Set CGRAM Address	0	0	0	1	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	37us
Set DDRAM Address	0	0	1	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	37us
Read Busy Flag & Address	0	1	BF	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	0us
Write Data	1	0	D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀	37us

- 표시는 0이든
1이든 상관 없
음 (don't care).

★ 유일한
Read 명령

Write 명령이 실행되는 동안
BF=1 이고, 실행이 끝나면 BF=0 이 됨.

I/D=0: DDRAM write/read 후 번지 감소.
I/D=1: DDRAM write/read 후 번지 증가.

S=0: DDRAM write 후 커서가 이동.
S=1: DDRAM write 후 화면이 이동.

D=0: Display Off
D=1: Display On

C=0: Cursor Off
C=1: Cursor On

B=0: Cursor Blinking Off
B=1: Cursor Blinking On

S/C=0: 커서가 한 자리 shift.
S/C=1: 화면이 한 자리 shift.

R/L=0: 왼쪽으로 shift.
R/L=1: 오른쪽으로 shift.

DL=0: 4-bit data 전송 (D4~D7)
DL=1: 8-bit data 전송 (D0~D7)

N=0: 1줄 표시
N=1: 2줄 표시

F=0: 5x8 font
F=1: 5x10 font

★ 명령어를 Write 한 후 Busy Flag (BF)가 0이 될 때까지 체크하는 것이 정석이나, BF를 체크하는 대신 Max 실행시간을 기다리는 방식으로 코드 작성.

이 경우 R/W 핀은 GND에 연결.

HD44780 Reset 상태 및 초기 설정 (8-bit)

Power-On Reset 후 상태

1. Display Clear.
2. Function set:
 - DL=1 (8-bit data)
 - N=0 (1-line display)
 - F=0 (5x8 font)
3. Display on/off control:
 - D=0 (Display off)
 - C=0 (Cursor off)
 - B=0 (Blinking off)
4. Entry mode set:
 - I/D=1 (Increment)
 - S=0 (No screen shift)

추가해야 할 초기화 명령어

Function Set

8-bit data
2-line display
5x8 font

Display on/off
control

Display On
Cursor Off
Blinking Off

I/O Port 핀 CPP Defines

회로의 핀을 변경하는 경우, 코드 상단 한 곳에서 Port와 핀 번호를 변경하여 이것이 전체 코드에 적용되도록 CPP (C Pre-Processor)의 #define directive를 사용하면 오류가 감소.

```
#define LCD_EN_DDR      DDRD
#define LCD_EN_PORT     PORTD
#define LCD_EN_BIT      4
```

```
#define LCD_RS_DDR      DDRD
#define LCD_RS_PORT     PORTD
#define LCD_RS_BIT      6
```

```
#define LCD_DB_DDR      DDRB
#define LCD_DB_PORT     PORTB
#define LCD_DB_BIT      0
```

HD44780 Low-Level 코드 (8-bit mode)

```
#define F_CPU 8000000
```

```
#include <avr/io.h>
```

```
#include <util/delay.h>
```

```
#define LCD_EN_DDR    DDRD
#define LCD_EN_PORT    PORTD
#define LCD_EN_BIT    4
```

```
#define LCD_RS_DDR    DDRD
#define LCD_RS_PORT    PORTD
#define LCD_RS_BIT    6
```

```
#define LCD_DB_DDR    DDRB
#define LCD_DB_PORT    PORTB
#define LCD_DB_BIT    0
```

```
static void lcd_write_byte(uint8_t rs, uint8_t data) {
```

```
    //// RS Pin 에 rs 값을 출력 (0 또는 1)
    //// E Pin 에 1 출력
    //// DB7~DB0 에 data 값 출력
    //// E Pin 에 0 출력
```

```
}
```

```
static void lcd_init() {
```

```
    //// RS, E Pin의 방향을 Output으로 설정
    //// DB7~DB0 Pin의 방향을 Output으로 설정
```

```
    _delay_ms(20);          // Power-on Reset이 실행되기 기다림.
```

```
    lcd_write_byte(0, ??); // 8-bit mode, 2 lines, 5x8 font
```

```
    _delay_us(40);
```

```
    lcd_write_byte(0, ??); // Display On, Cursor Off, Blinking Off
```

```
    _delay_us(40);
```

```
}
```

//// 구문과 ?? 항목을
완성할 것.

HD44780 High-Level 코드

```
static void lcd_puts(char * str) {
    uint8_t i;

    lcd_write_byte(0, 0x80);      // move to 1st line
    _delay_us(40);

    for (i=0; i<16; i++) {
        if (str[i] == '\0')
            return;
        lcd_write_byte(1, str[i]);
        _delay_us(40);
    }

    lcd_write_byte(0, 0x80+0x40); // move to 2nd line
    _delay_us(40);

    for (i=16; i<32; i++) {
        if (str[i] == '\0')
            return;
        lcd_write_byte(1, str[i]);
        _delay_us(40);
    }
}
```

```
int main(void) {
    uint8_t i=0;
    char message[] = "00 seconds";

    lcd_init();
    lcd_puts("0123456789ABCDEF!@#$$%^&*()abcdef");
    _delay_ms(2000);

    lcd_write_byte(0, 0x01); // display clear
    _delay_ms(2);

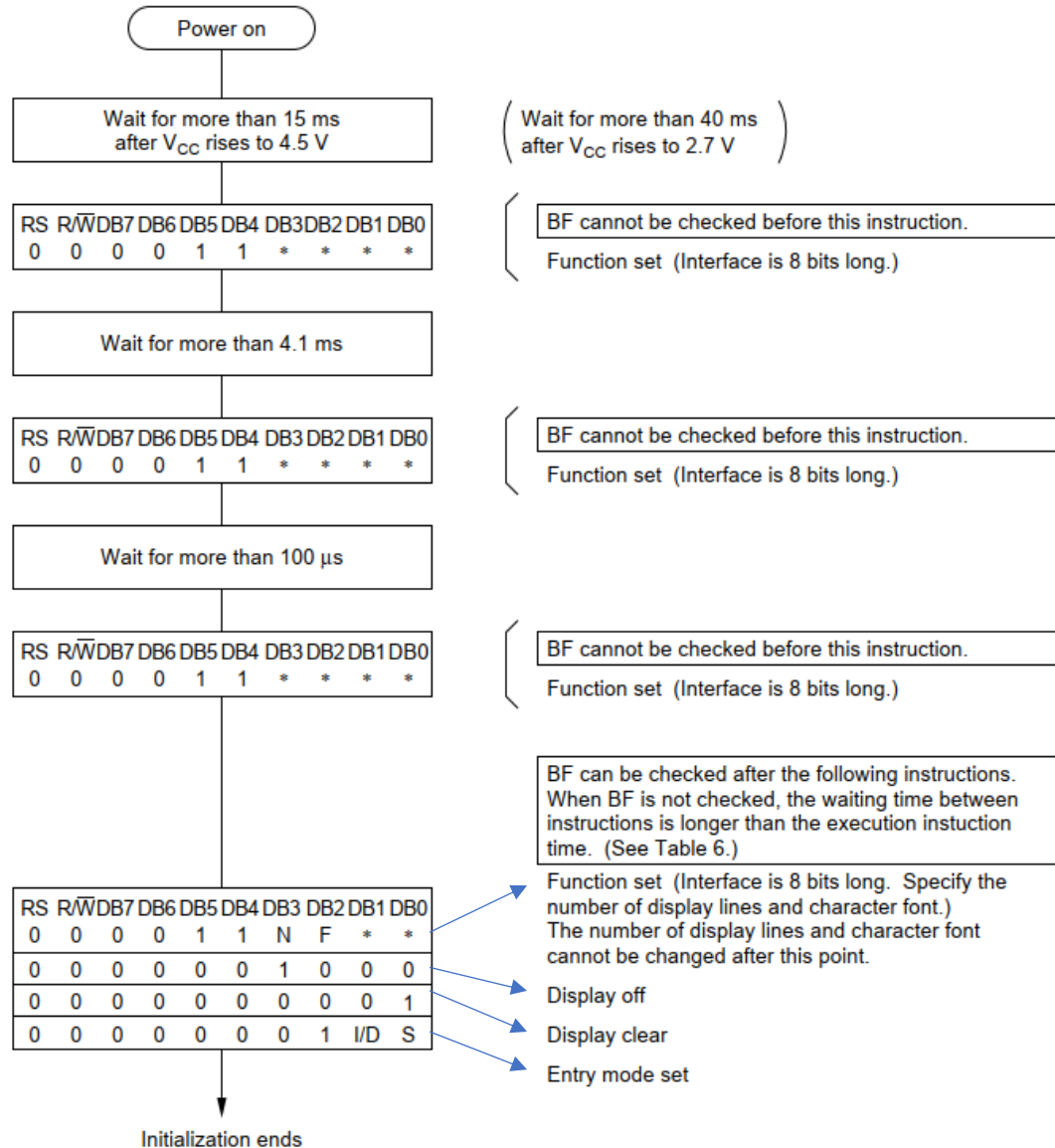
    while (1) {
        message[0] = i/10+'0';
        message[1] = i%10+'0';
        lcd_puts(message);
        if (++i>=100)
            i=0;
        _delay_ms(1000);
    }
}
```


HD44780 동작 동영상



Power-On Reset 이 안 되는 경우

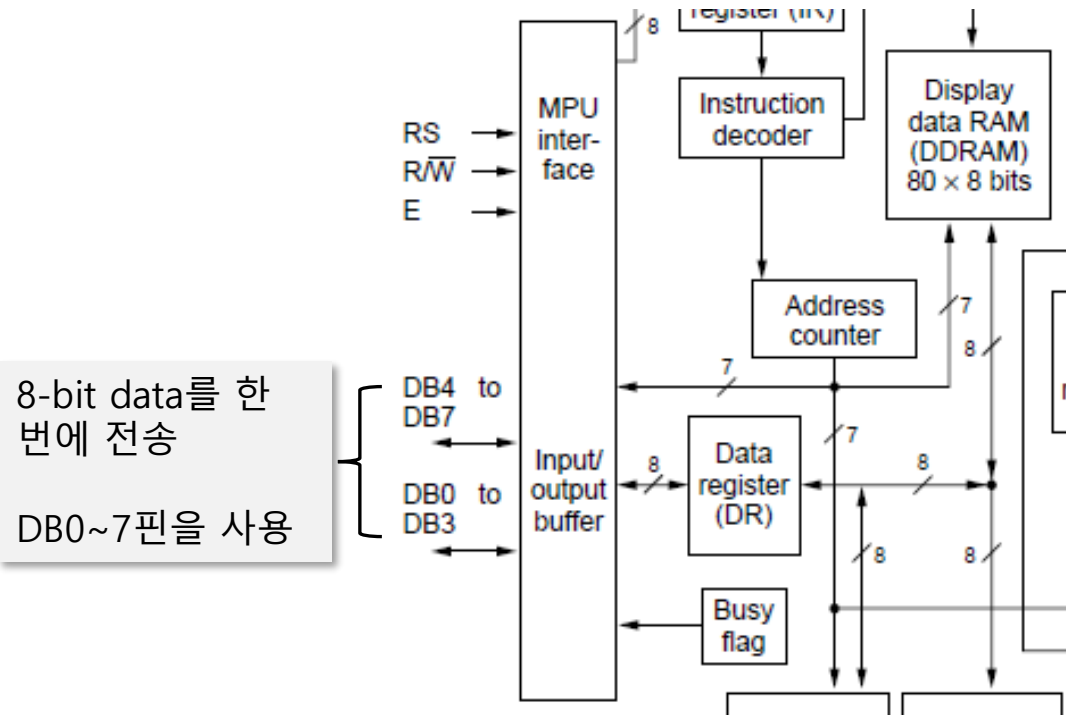
Datasheet 45쪽



`lcd_init()` 함수를 이 순서와 값으로 수정.

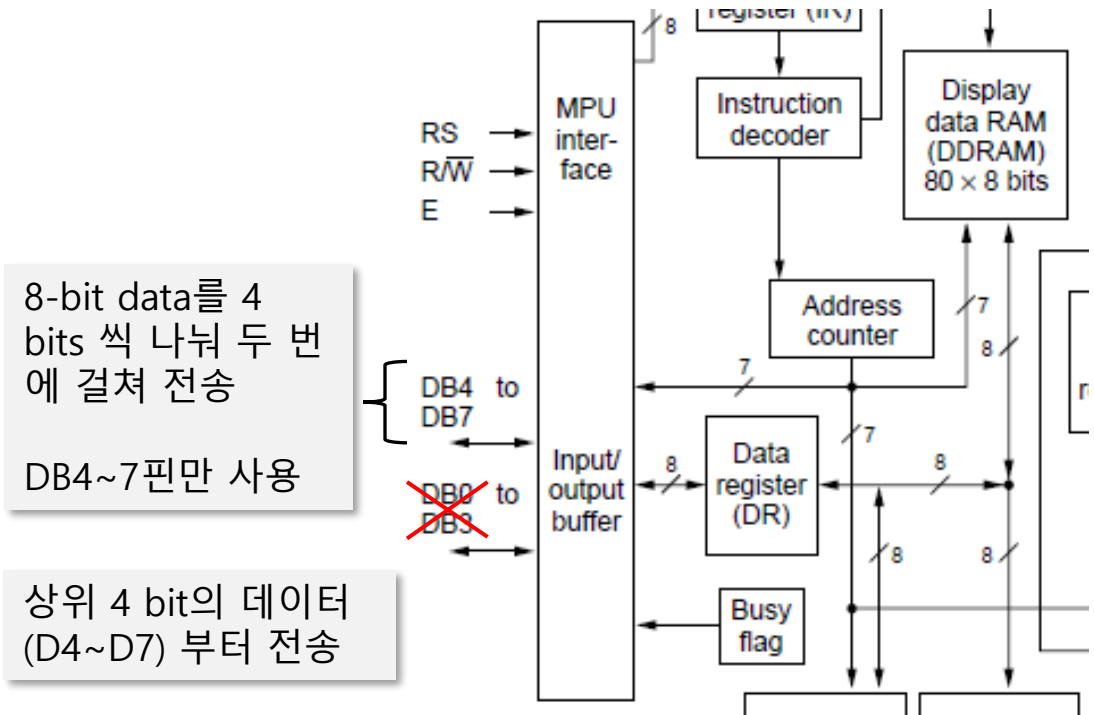
HD44780 8-bit vs. 4-bit Interface

8-bit Interface (DL=1)



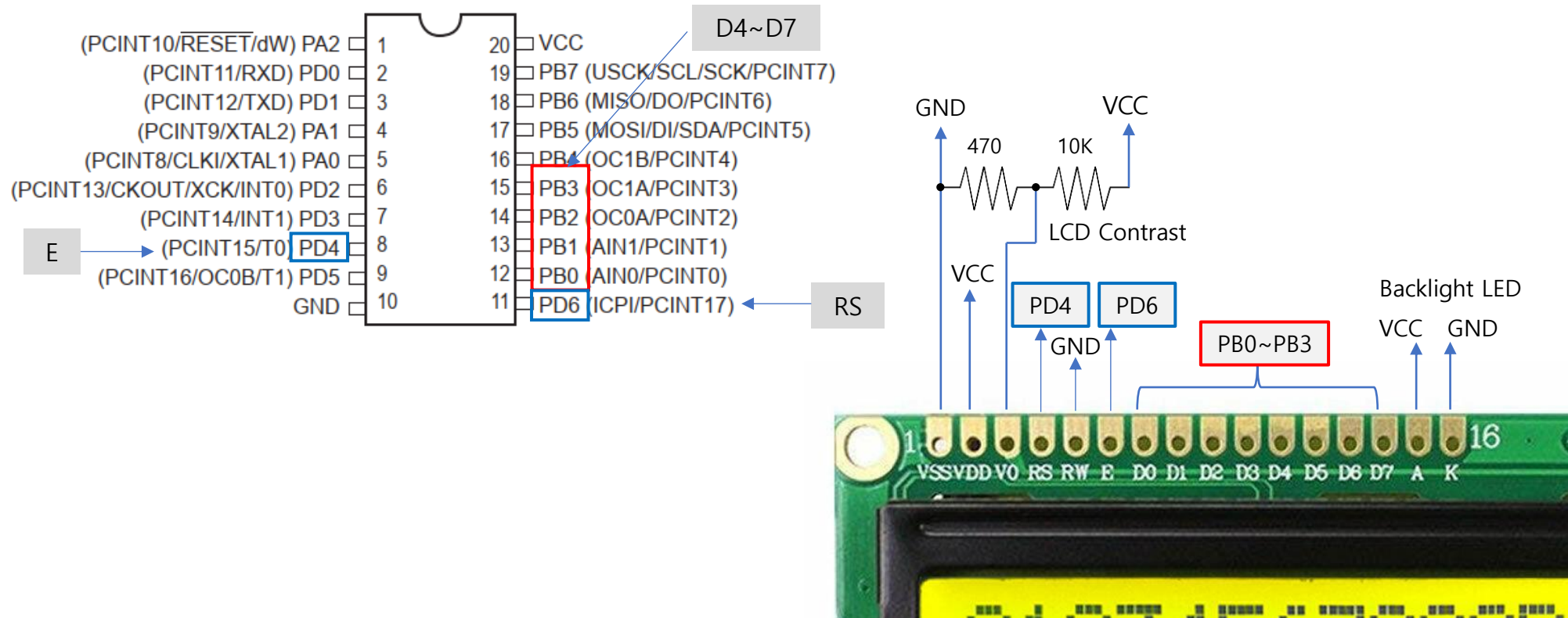
Datasheet 32쪽

4-bit Interface (DL=0)



Datasheet 33쪽

HD44780 – ATtiny 연결 (4-bit)



HD44780 Low-Level 코드 (4-bit)

```
static void lcd_write_nibble(uint8_t rs, uint8_t data) {  
    //// RS Pin 에 rs 값을 출력 (0 또는 1)  
    //// E Pin 에 1 출력  
    //// DB3~0 Pin 에 변수 data의 Bit 3~0를 출력  
    //// E Pin 에 0 출력  
}  
  
static void lcd_write_byte(uint8_t rs, uint8_t data) {  
    //// 변수 data의 Bit 7~4을 lcd_write_nibble()로 보냄.  
    //// 변수 data의 Bit 3~0을 lcd_write_nibble()로 보냄.  
}
```

Datasheet 46쪽
참조

```
static void lcd_init() {  
    //// RS, E Pin의 방향을 Output으로 설정  
    //// DB3~DB0 Pin의 방향을 Output으로 설정  
  
    _delay_ms(20);  
  
    lcd_write_nibble(0, 0x3); // Function Set: 8-bit mode  
    _delay_ms(5);  
    lcd_write_nibble(0, 0x3);  
    _delay_us(100);  
    lcd_write_nibble(0, 0x3);  
    _delay_us(40);  
    lcd_write_nibble(0, 0x2); // Function Set: 4-bit mode  
    _delay_us(40);  
    lcd_write_byte(0, 0x28); // Function Set: 4-bit, 2 lines, 5x8 font  
    _delay_us(40);  
    lcd_write_byte(0, 0x08); // Display Off  
    _delay_us(40);  
    lcd_write_byte(0, 0x01); // Display clear  
    _delay_ms(2);  
    lcd_write_byte(0, 0x06); // Entry mode = increment, no screen shift  
    _delay_us(40);  
    lcd_write_byte(0, 0x0c); // Display On  
    _delay_us(40);  
}
```

HD44780 CGRAM (사용자 폰트 영역)

8개의 사용자 font를 CGRAM에 저장할 수 있음.

CGRAM은 6 bits 의 주소로 지정함.

- 상위 3 bits 은 8개의 font 중 하나를 선택.
- 하위 3 bits 은 각 font의 8개 row 중 하나를 선택.

Font의 한 row 는 5 bits 로 구성됨.

- Bit=1 이면 font의 점이 On, 0 이면 Off.
- 상위 3 bits은 사용하지 않음.

명령어 0b01XXXXXX 로 CGRAM 주소를 지정함.

Instruction	RS	R/W	Code							
			DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
Set CGRAM address	0	0	0	1	ACG	ACG	ACG	ACG	ACG	ACG

사용자 폰트 0번

사용자 폰트 1번



사용자 폰트 7번

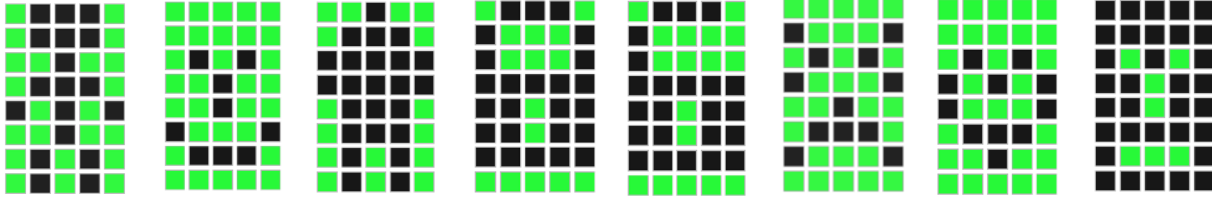
CGRAM Address						Character Patterns (CGRAM data)							
5	4	3	2	1	0	7	6	5	4	3	2	1	0
High			Low			High			Low				
0	0	0	0	0	0	*	*	*	1	1	1	1	0
			0	0	1	↑			1	0	0	0	1
			0	1	0				1	0	0	0	1
			0	1	1				1	1	1	1	0
			1	0	0				1	0	1	0	0
			1	0	1				1	0	0	1	0
			1	1	0				1	0	0	0	1
			1	1	1	*	*	*	0	0	0	0	0
0	0	1	0	0	0	*	*	*	1	0	0	0	1
			0	0	1	↑			0	1	0	1	0
			0	1	0				1	1	1	1	1
			0	1	1				0	0	1	0	0
			1	0	0				1	1	1	1	1
			1	0	1				0	0	1	0	0
			1	1	0				0	0	1	0	0
			1	1	1				*	*	*	0	0
			0	0	0	*	*	*					
			0	0	1	↑							
1	1	1	1	0	0				↓				
			1	0	1								
			1	1	0								
			1	1	1	*	*	*					

사용자 폰트에는 문자 Code 0x00 ~ 0x07 가 부여됨.

	Upper 4 Bits		Lower 4 Bits					
	0000	0001	0010	0011	0100	0101		
xxxx0000	CG RAM (1)			0	2	P		
xxxx0001	(2)		!	1	A	Q		
xxxx0010	(3)		"	2	B	R		
xxxx0011	(4)		#	3	C	S		
xxxx0100	(5)		\$	4	D	T		
xxxx0101	(6)		%	5	E	U		
xxxx0110	(7)		&	6	F	V		
xxxx0111	(8)		'	7	G	W		

CGRAM 사용 예제

<https://circuitdigest.com/microcontroller-projects/custom-characters-on-lcd-using-pic16f877a>



Font Data

```
const uint8_t custom5x8[] = {
    0b01110,0b01110,0b00100,0b01110,0b10101,0b00100,0b01010,0b01010, // Font for CGRAM Address 0x00 (문자 Code 0)
    0b00000,0b00000,0b01010,0b00100,0b00100,0b10001,0b01110,0b00000, // Font for CGRAM Address 0x08 (문자 Code 1)
    0b00100,0b01110,0b11111,0b11111,0b01110,0b01110,0b01010,0b01010, // Font for CGRAM Address 0x10 (문자 Code 2)
    0b01110,0b10001,0b10001,0b11111,0b11011,0b11011,0b11111,0b00000, // Font for CGRAM Address 0x18 (문자 Code 3)
    0b01110,0b10000,0b10000,0b11111,0b11011,0b11011,0b11111,0b00000, // Font for CGRAM Address 0x20 (문자 Code 4)
    0b00000,0b10001,0b01010,0b10001,0b00100,0b01110,0b10001,0b00000, // Font for CGRAM Address 0x28 (문자 Code 5)
    0b00000,0b00000,0b01010,0b10101,0b10001,0b01110,0b00100,0b00000, // Font for CGRAM Address 0x30 (문자 Code 6)
    0b11111,0b11111,0b10101,0b11011,0b11011,0b11111,0b10001,0b11111 // Font for CGRAM Address 0x38 (문자 Code 7)
};
```

CGRAM에 Font 저장

```
//// 저장할 CGRAM 주소를 Set CGRAM Address 명령어로 보냄.
_delay_us(40);

for (uint8_t i=0; i<sizeof(custom5x8); i++) {
    //// custom5x8[i] 값을 읽어서 Write Data 명령어로 보냄.
    _delay_us(40);
}
```

CGRAM Font를 표시

```
//// 표시하고자 하는 화면 위치를 Set DDRAM Address 명령어로 보냄.
_delay_us(40);

//// 표시하고자 하는 문자 Code (0~7)을 Write Data 명령어로 보냄.
_delay_us(40);
```


[과제 1] HD44780 응용

1. 8-bit mode로 문자 출력
 2. 사용자 Font를 포함한 4-bit mode 문자 출력
- 실습 예제를 참조하여 각자 창의적인 응용을 구현.
 - 제출물: 회로도, 코드, 동작 동영상
 - 회로도는 손으로 작성하여도 무방
 - 회로도 CAD로는 EasyEDA, KiCAD 등이 있음.
 - ZIP 파일로 압축하여 eTL에 업로드
 - ZIP 파일 이름은 본인이름_학교_과제1.ZIP
 - 마감: 9월 16일 (수) 11:59 PM
 - 지각 제출도 가능하나 감점함.